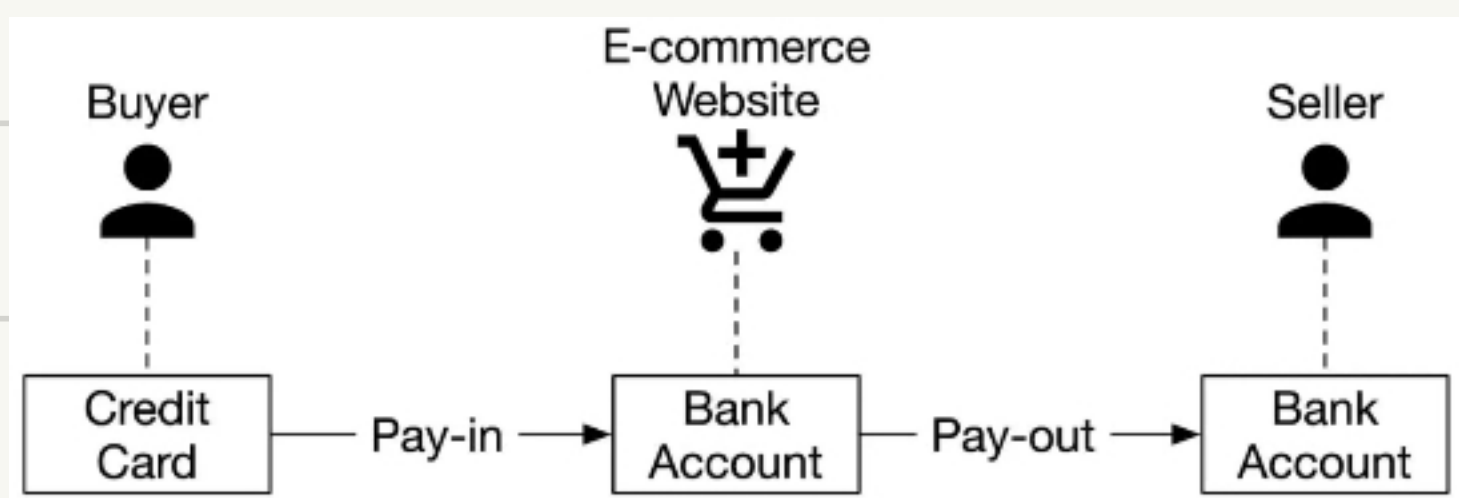
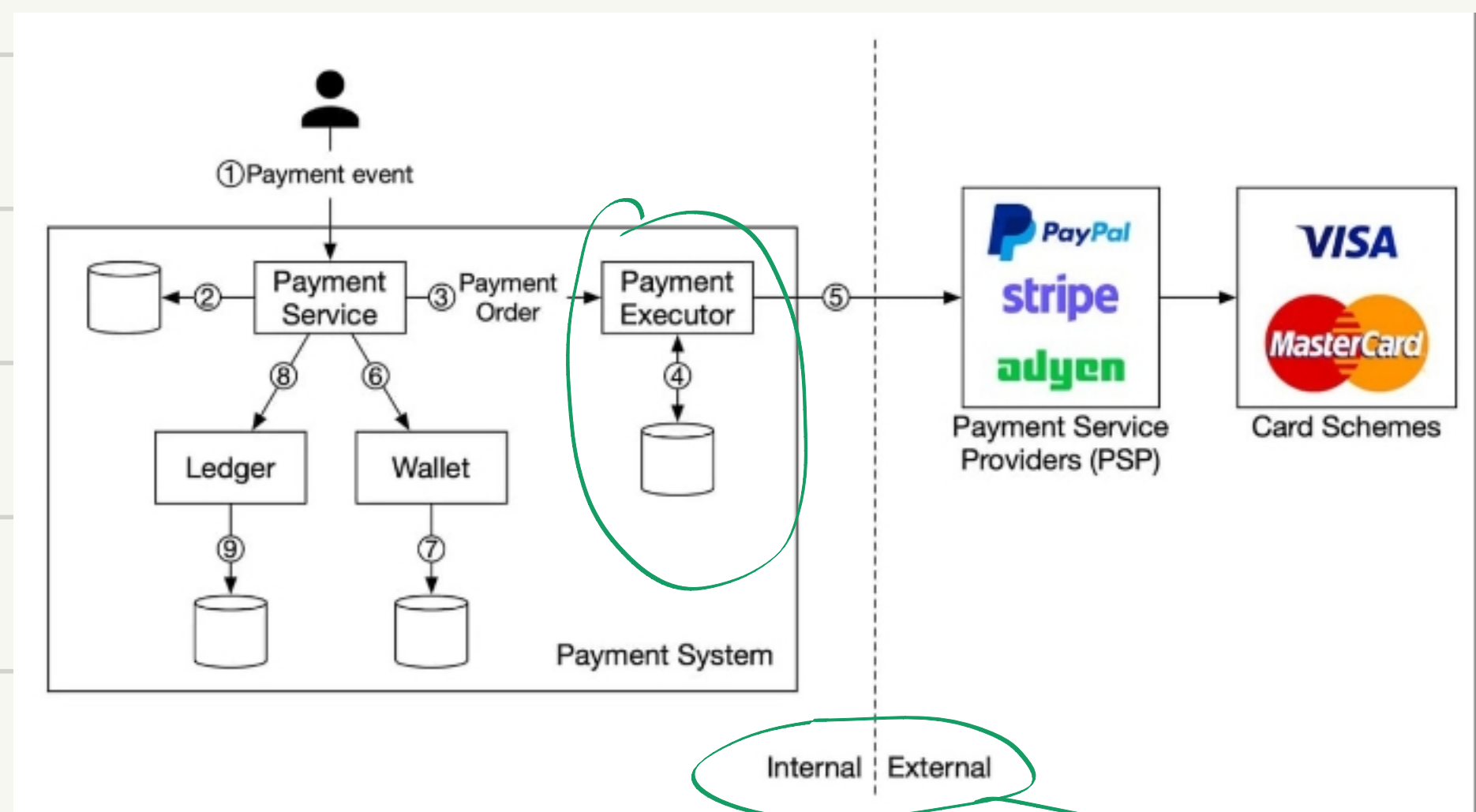


REAL WORLD PAYMENT SERVICES

(TRACK-3)



In the
payment process



(Source)

<https://newsletter.pragmaticengineer.com/p/designing-a-payment-system>

Payment (much like OS level thread) is a state-entity problem, and any single glitch in any entities can lead to a transaction failing

Failures - <https://razorpay.com/blog/online-payments-failure-reasons/>

→ And any error in this is not a single monolith error, most of the times, it is a process involving different services, components, gateways, etc.

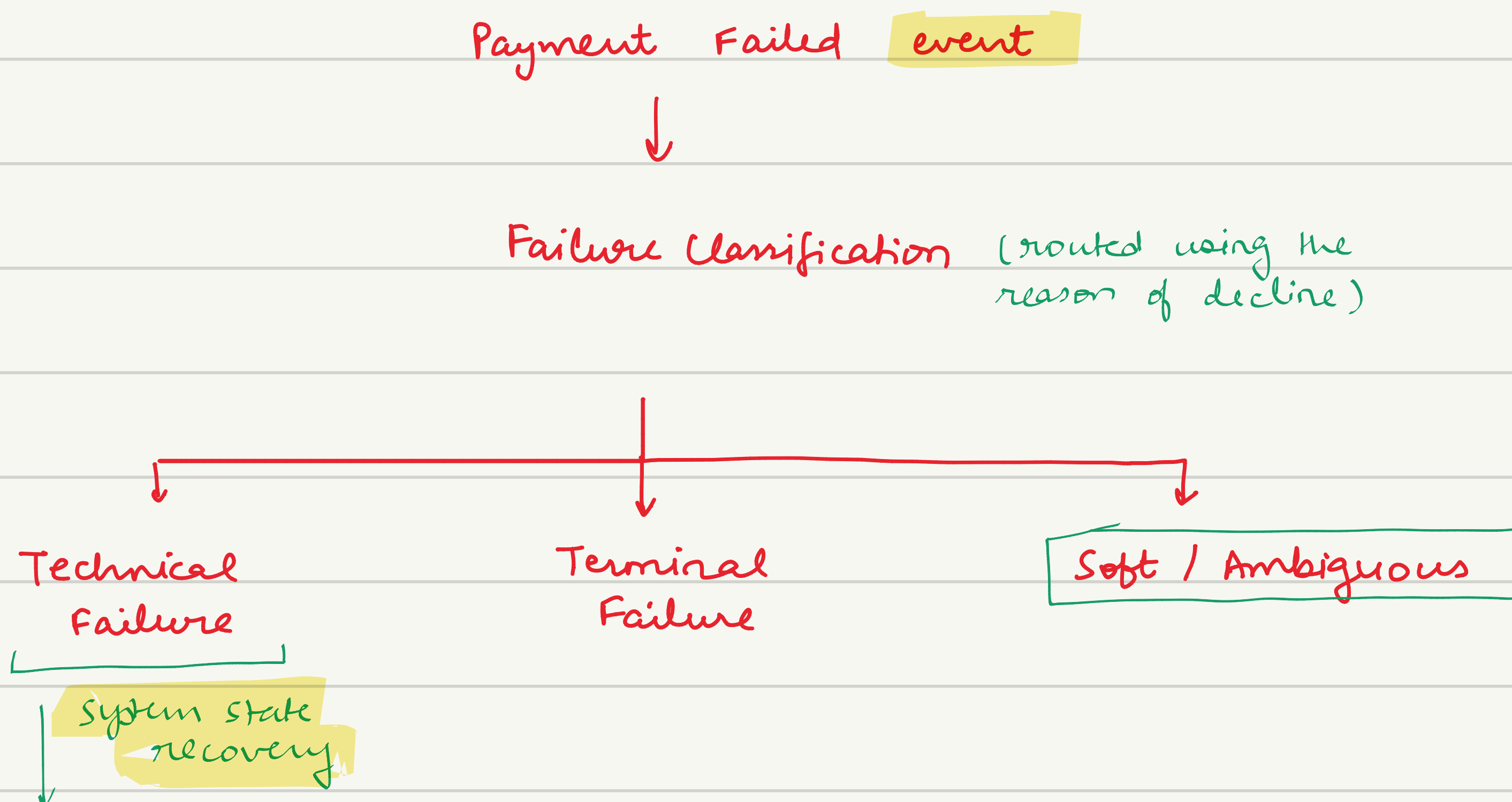
↓
NW, scale/amount of transactions/sec

↓
may be other than current (Microservices)

→ Diverse causes of payment failure creates a need to study, classify them and monitor different recovery strategies

Agents Recovering Payment Failures

can an agent help us to recover our revenue
from all kinds of payment failures?



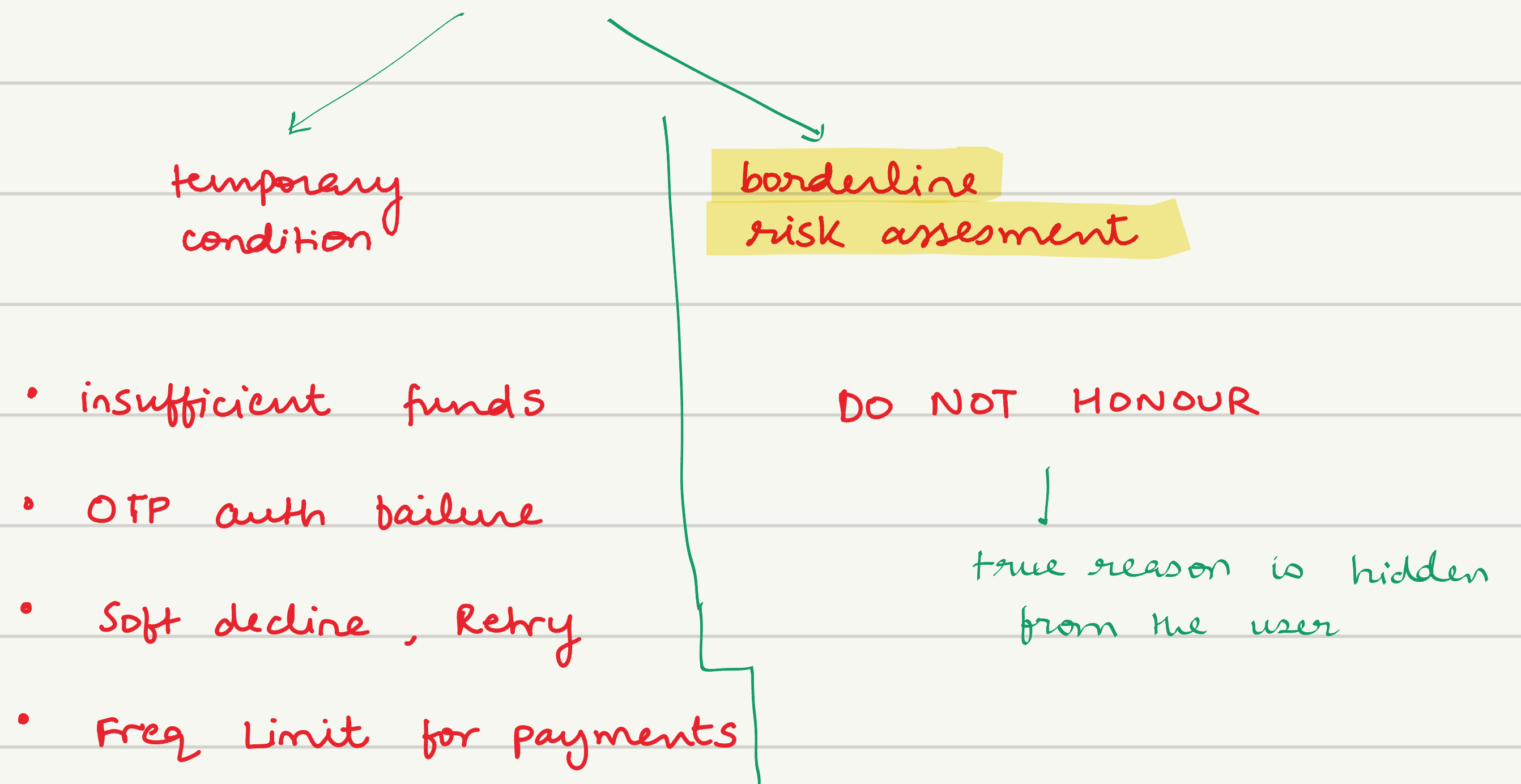
These are service / system failures while the request was passing through different services (microservice ecosystem)

- Payment Gateway Failure / Down / 5XX Internal (Saga Choreography)
- Double payment (duplicate) → Redis based Idempotency
- Network blw services ↓ (traffic)
- Banking system timeout (Saga)
- Message Queue down (OutBox Pattern)
- Gateway Rate Limiting (429) (Redis)

Terminal Failure : Payment processing outcome where the transaction fails due to a permanent condition that can't be resolved by retrying the payment

- Card expired / lost / stolen
- Account closed
- Fraud block for a user

Soft Failure : Payment processing outcome where transaction is rejected due to a

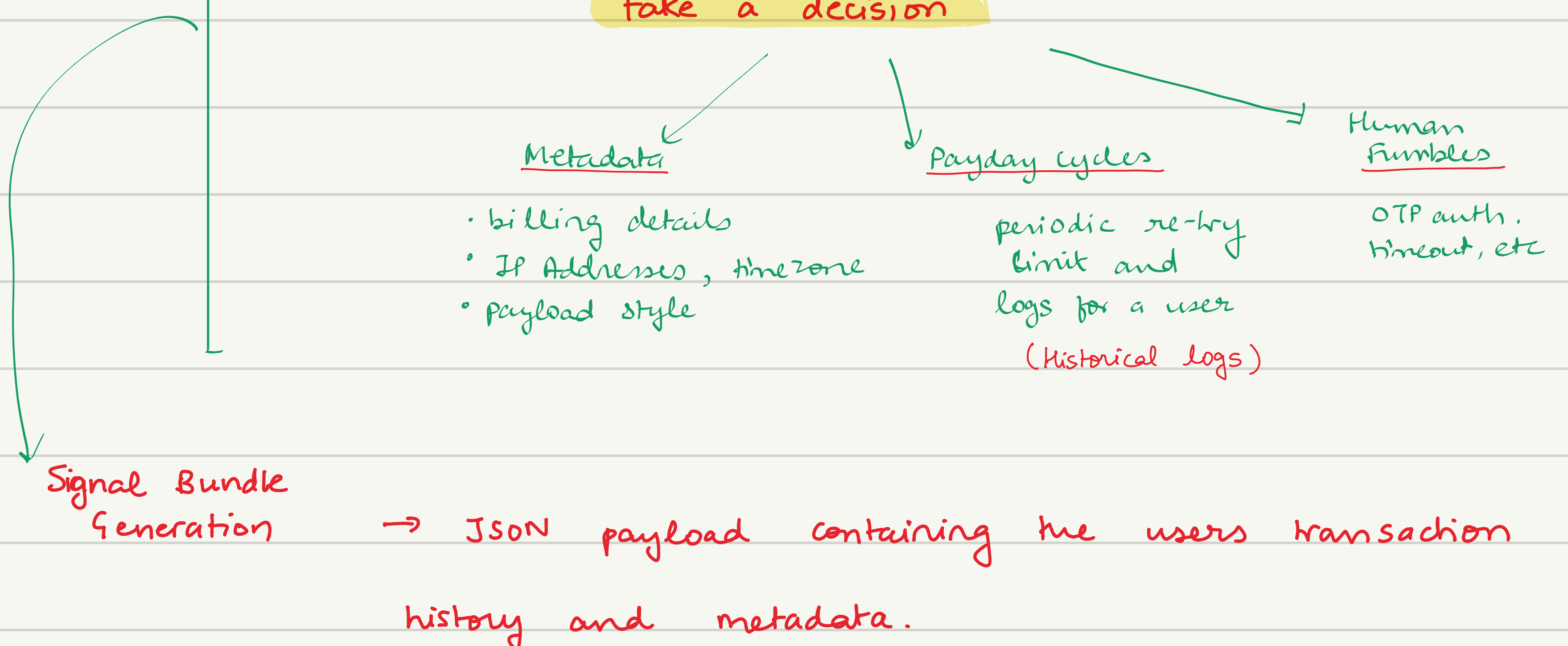


→ These are the specific errors where the money that has left the system / service is recoverable by using an Agent.

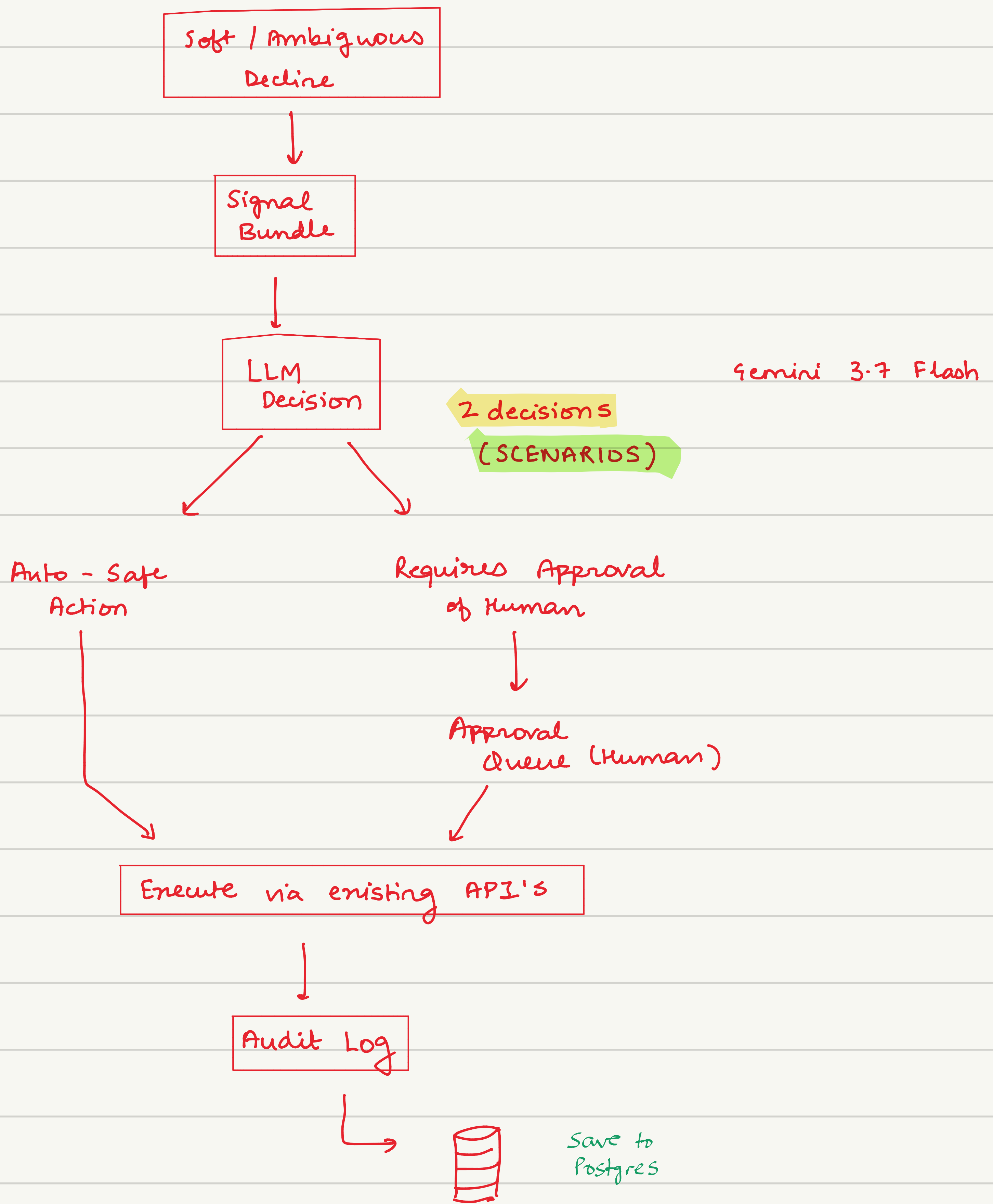
Because they are

- content dependent
- we need to look at both external factors and patterns / guidelines to

take a decision



→ We can specifically check these failures in form of stimulated payments at scale to see how well the system is able to recover revenue for different issues.

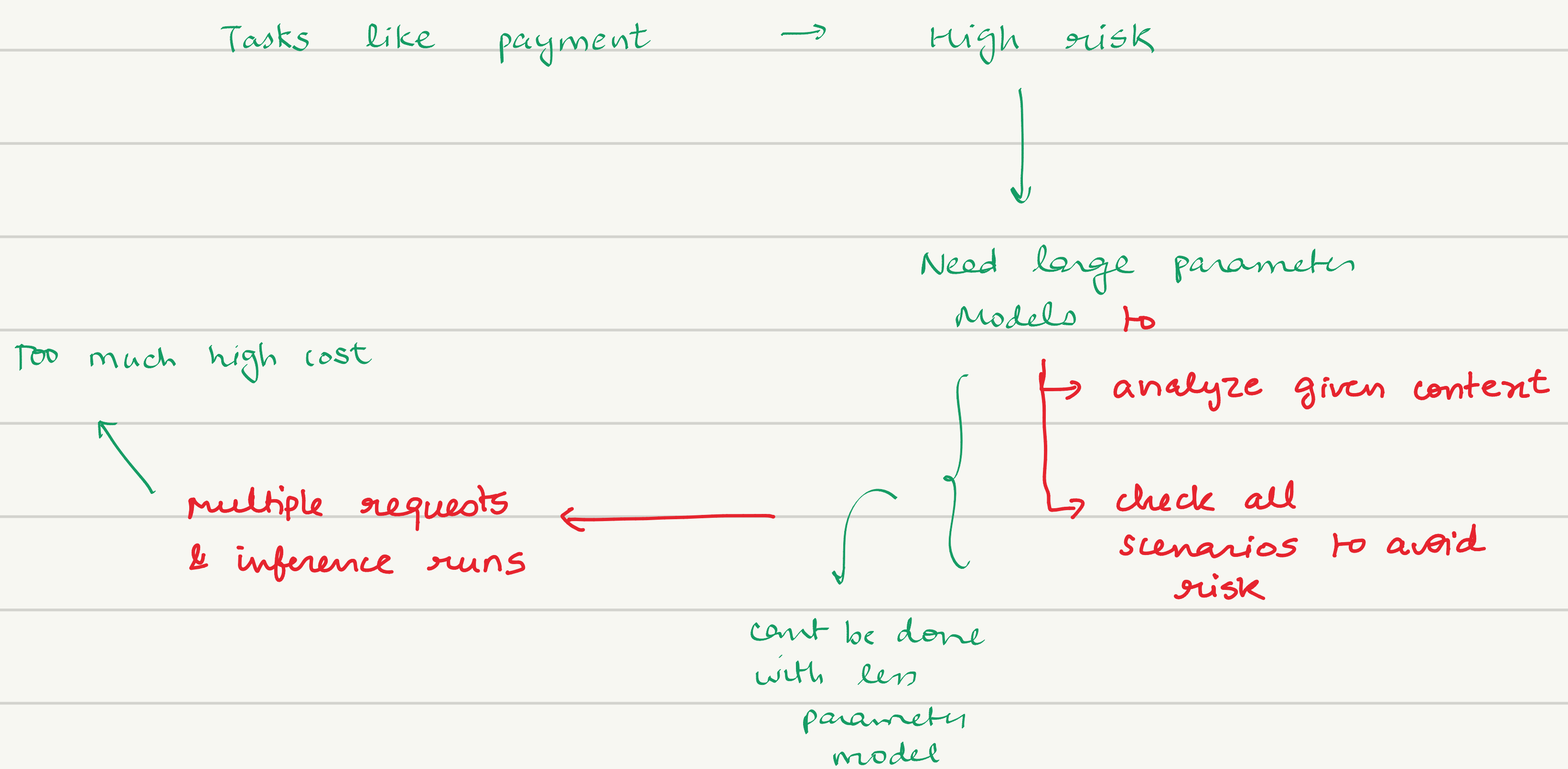


How we decided the scenarios ?

Scenario A	Scenario B
• High trust but insufficient bunds • Limits (payment gateway) • Soft decline → retry later • (Do not honour) with history	cases of Low confidence from LLM + High financial / marginal risk Do not honour w/o history OTP Auth failure

→ You can't fire too many requests to a production LLM, because of its rate limits. So, a problem @ a lower level, as we wish to scale the application.

→ System may be needs its local LLM to process that many requests at a time, and smaller LLM's (with less parameters) are not massively trustworthy to take decisions with all the content for.



→ Gemini 3.7 Flash keeps things in proper check, being a bigger model than smaller / local models that may lack complex contextual understanding to distinguish between temporary gateway limit & a high risk fraud.